

PRUEBA TÉCNICA

Desarrollador Fullstack Junior/Mid

Sector Logística • NestJS + Angular + PostgreSQL

Posición	Desarrollador Fullstack Junior/Mid
Sector	Logística / Supply Chain
Stack	NestJS (Backend) + Angular (Frontend) + PostgreSQL
Duración estimada	4 – 5 horas (entrega en 72 horas)
Formato de entrega	Repositorio Git (GitHub/GitLab)

Instrucciones Generales

1. Lee toda la prueba antes de comenzar.
2. Crea un monorepo o dos repositorios separados (backend y frontend).
3. Incluye un README.md con instrucciones para levantar el proyecto.
4. Usa Git con commits descriptivos.
5. Docker Compose es opcional pero valorado.
6. Utiliza typescript como lenguaje de desarrollo. Se valorará el uso de tipos.

Contexto del Proyecto

Una empresa de logística necesita un sistema interno para registrar envíos, actualizar su estado a lo largo de la cadena logística, y permitir que los clientes consulten el estado de sus paquetes mediante un código de seguimiento.

Flujo de estados:

```
CREADO → EN_ALMACÉN → EN_TRÁNSITO → EN_REPARTO → ENTREGADO
                                     ↳ DEVUELTO
(cualquier estado) → CANCELADO
```

1 Backend (NestJS)

1.1 Descripción

Se espera que construyas una API REST con NestJS y PostgreSQL que dé soporte a la operativa diaria de TransLog. La API será consumida tanto por la aplicación Angular de los operadores como por la vista pública de tracking para clientes finales.

El backend debe cubrir tres áreas funcionales principales:

Autenticación y usuarios. El sistema debe permitir el registro y login de usuarios mediante JWT. Cada usuario tiene un rol (operador o supervisor) que determina sus permisos. Los supervisores pueden registrar nuevos usuarios; los operadores solo pueden autenticarse y trabajar con envíos.

Gestión de envíos. Los operadores autenticados deben poder crear nuevos envíos indicando las direcciones de origen y destino, el nombre del destinatario, un teléfono de contacto opcional y el peso del paquete en kilogramos. Al crearse, el sistema debe generar automáticamente un código de seguimiento único con formato `ENV-YYYYMMDD-XXXX`. Debe ser posible listar los envíos con paginación y filtro por estado, consultar el detalle de un envío incluyendo su historial completo de eventos, y cancelar un envío siempre que no haya sido entregado previamente. La funcionalidad central es el cambio de estado: cada transición debe respetar el flujo lógico definido, registrarse como un evento de seguimiento con fecha, usuario responsable, ubicación y notas, y al marcarse como entregado, registrar automáticamente la fecha de entrega.

Tracking público. Un endpoint sin autenticación que permita consultar el estado actual y el historial de un envío a partir de su código de seguimiento. Este endpoint será consumido por la vista pública del frontend.

El diseño del modelo de datos es responsabilidad del candidato. Se valorará que sea coherente, esté normalizado y refleje correctamente el dominio del negocio.

1.2 Endpoints

Autenticación

- `POST /auth/register` y `POST /auth/login` (JWT)

Envíos (requieren autenticación)

- `POST /shipments` — Crear envío (genera `trackingCode`)
- `GET /shipments` — Listar con paginación y filtro por estado
- `GET /shipments/:id` — Detalle con historial de eventos
- `PATCH /shipments/:id/status` — Cambiar estado (valida transiciones, crea `ShipmentEvent`)
- `DELETE /shipments/:id` — Cancelar (solo si no está ENTREGADO)

Tracking público (sin autenticación)

- `GET /tracking/:trackingCode` — Estado e historial por código de seguimiento

1.3 Reglas de Negocio

- Las transiciones de estado deben respetar el flujo definido.
- Cada cambio de estado genera un `ShipmentEvent` con timestamp, ubicación y usuario.
- Al marcar `DELIVERED` se registra automáticamente `deliveredAt`.
- No se puede cancelar un envío ya entregado.
- Solo `SUPERVISOR` puede registrar nuevos usuarios.

1.4 Requisitos Técnicos

- TypeORM o Prisma con PostgreSQL.
- Validación de DTOs con `class-validator`.
- Guard de autenticación (JWT) y guard de rol.
- Exception Filter global.
- Swagger básico (decoradores en controllers).
- Al menos 2 tests unitarios de la lógica de negocio.

1.5 Desafío Algorítmico: Asignación de Carga

Este ejercicio evalúa tu capacidad de resolver un problema lógico con estructuras de datos básicas. **Debe implementarse como un endpoint funcional del API.**

Endpoint: `POST /shipments/assign-vehicles`

Contexto:

La empresa tiene vehículos de reparto con una capacidad máxima de carga (kg). Se necesita distribuir un conjunto de envíos en el menor número de vehículos posible.

Entrada:

```
{
  "shipmentIds": ["uuid-1", "uuid-2", "uuid-3", ...],
  "vehicleCapacity": 100
}
```

Ejemplo de salida esperada:

```
{
  "vehicles": [
    {
      "vehicleNumber": 1,
      "shipments": [
        { "shipmentId": "uuid-4", "trackingCode": "ENV-...", "weight": 45.0 },
        { "shipmentId": "uuid-1", "trackingCode": "ENV-...", "weight": 32.5 }
      ]
    }
  ]
}
```

```
      "totalWeight": 77.5,  
      "remainingCapacity": 22.5  
    },  
    { "vehicleNumber": 2, ... }  
  ],  
  "totalVehiclesUsed": 3,  
  "totalWeight": 267.5  
}
```

Requisitos:

1. Implementar First Fit Decreasing: ordenar envíos por peso descendente y asignar cada uno al primer vehículo donde quepa.
2. Obtener los pesos reales desde la base de datos.
3. Validar que todos los shipmentIds existan y estén en estado IN_WAREHOUSE.
4. Si un envío excede la capacidad del vehículo, retornar un error descriptivo.
5. Incluir al menos 1 test unitario específico para la lógica del algoritmo.

💡 Este problema se conoce como Bin Packing. No buscamos la solución óptima: evaluamos que puedas descomponer el problema, elegir una estructura de datos y escribir código limpio y testeable.

2 Frontend (Angular)

2.1 Configuración

- Angular v17+ con standalone components.
- Angular Material o PrimeNG.
- Lazy loading en rutas.

2.2 Vistas Requeridas

Login y Registro

- Formularios con validaciones reactivas.
- Gestión de sesión (JWT en storage), Route Guard e Interceptor HTTP.

Listado de Envíos

- Tabla con: código de seguimiento, destinatario, destino, estado (badge de color), fecha.
- Paginación servidor y filtro por estado.
- Botón para crear nuevo envío (formulario en modal o página).

Detalle del Envío

- Datos del envío y timeline/stepper con el historial de eventos.
- Acción para cambiar estado (solo transiciones válidas) con campo de ubicación y notas.

Tracking Público

- Página sin login: el usuario ingresa su código y ve el estado actual e historial.

2.3 Requisitos Técnicos

- Reactive Forms con validaciones.
- Servicios para comunicación con el API.
- Loading states y manejo de errores con feedback (toasts/snackbar).
- Estructura modular con standalone components.

Puntos Bonus (Opcionales)

No obligatorios, pero demuestran un nivel superior:

- **Docker Compose** que levante todo con un solo comando.
- **Vista de asignación de vehículos** en el frontend: seleccionar envíos, ejecutar el algoritmo y mostrar resultado con barras de capacidad.
- **Dashboard básico** (solo SUPERVISOR): cards con total de envíos por estado.

- **Exportación CSV** del listado de envíos.
 - **CI/CD** pipeline básico con GitHub Actions (lint + test).
 - Integración de **Swagger** en el backend.
-

Formato de Entrega

1. Repositorio Git con acceso al evaluador.
 2. **README.md** con: descripción del proyecto, decisiones técnicas, explicación breve del algoritmo implementado e instrucciones para levantar el entorno.
 3. Historial de commits limpio y descriptivo.
 4. Si usas Docker, debe levantarse con un solo comando.
-

¡**Mucha suerte!** Si tienes dudas sobre los requerimientos, consúltalas antes de comenzar.